RMAN Recovery Manager

Complete Guide to Backup, Copy, Recovery & Flashback Operations

RMAN COPY Operations

RMAN COPY creates image copies of datafiles, control files, and archived redo logs. Unlike backupsets, image copies are identical byte-for-byte copies that can be used directly by the database.

Key Benefits:

- Faster recovery than backupsets (no restore needed, just switch)
- Can be incrementally updated to reduce recovery time
- Usable by OS utilities (cp, dd, etc.)

Basic COPY Commands

```
# Copy entire database
RMAN> COPY DATABASE;

# Copy specific datafile
RMAN> COPY DATAFILE '/u01/oradata/ORCL/users01.dbf'
      TO '/backup/users01_copy.dbf';

# Copy tablespace
RMAN> COPY TABLESPACE users
      TO '/backup';

# Copy current control file
RMAN> COPY CURRENT CONTROLFILE TO '/backup/control01.ctl';
```

```
# Copy with tag for identification
RMAN> COPY DATAFILE 5 TO '/backup/df5.dbf' TAG 'WEEKLY_COPY';
```

Incremental Update of Image Copies

This powerful feature keeps image copies current, dramatically reducing recovery time.

```
# Step 1: Create initial image copy
RMAN> BACKUP AS COPY DATABASE;

# Step 2: Daily - Take incremental backup and apply to image copy
RMAN> BACKUP INCREMENTAL LEVEL 1
      FOR RECOVER OF COPY WITH TAG 'DAILY_INC'
      DATABASE;

# Step 3: Apply incremental to image copy (roll forward)
RMAN> RECOVER COPY OF DATABASE WITH TAG 'DAILY_INC';

# Combined approach (automatic)
RMAN> RUN {
  RECOVER COPY OF DATABASE WITH TAG 'INC_UPDATE';
  BACKUP INCREMENTAL LEVEL 1 FOR RECOVER OF COPY WITH TAG 'INC_UPDATE'
  DATABASE;
}
```

 Best Practice: Use incremental update strategy for databases requiring minimal downtime. Recovery time is reduced to just the time since last incremental update (typically 24 hours).

Database Recovery

Complete Recovery (No Data Loss)

Restore and recover database to the most recent point in time using all available redo.

```
# Recover entire database (database must be mounted)
RMAN> STARTUP MOUNT;
RMAN> RESTORE DATABASE;
RMAN> RECOVER DATABASE;
RMAN> ALTER DATABASE OPEN;

# Recover specific tablespace (database open, tablespace offline)
RMAN> SQL 'ALTER TABLESPACE users OFFLINE IMMEDIATE';
RMAN> RESTORE TABLESPACE users;
RMAN> RECOVER TABLESPACE users;
RMAN> SQL 'ALTER TABLESPACE users ONLINE';

# Recover specific datafile
RMAN> SQL 'ALTER DATABASE DATAFILE 5 OFFLINE';
RMAN> RESTORE DATAFILE 5;
RMAN> RECOVER DATAFILE 5;
RMAN> SQL 'ALTER DATABASE DATAFILE 5 ONLINE';

# Validate before actual restore
RMAN> RESTORE DATABASE VALIDATE;
```

Fast Recovery Using Image Copies (SWITCH)

```
# Switch datafile to image copy (instant recovery)
RMAN> SWITCH DATAFILE 5 TO COPY;

# Switch entire database to image copies
RMAN> SWITCH DATABASE TO COPY;

# Then apply redo logs to roll forward
RMAN> RECOVER DATABASE;
RMAN> ALTER DATABASE OPEN;
```

⚠ **Important:** SWITCH changes the control file pointer to use the image copy as the new datafile location. This is much faster than RESTORE but requires a valid image copy.

Point-in-Time Recovery (PITR)

PITR allows you to recover the database to a specific moment in the past, useful for:

- Recovering from logical corruption or user errors
- Testing backup and recovery procedures
- Recovering dropped tables or incorrect DML operations

Complete Database PITR

```
# Recover to specific time
RMAN> STARTUP MOUNT;
RMAN> SET UNTIL TIME "TO_DATE('2024-02-10 15:30:00', 'YYYY-MM-DD HH24:
RMAN> RESTORE DATABASE;
RMAN> RECOVER DATABASE;
RMAN> ALTER DATABASE OPEN RESETLOGS;

# Recover to SCN (System Change Number)
RMAN> RUN {
  SET UNTIL SCN 2894561;
  RESTORE DATABASE;
  RECOVER DATABASE;
  ALTER DATABASE OPEN RESETLOGS;
}

# Recover to log sequence
RMAN> RUN {
  SET UNTIL SEQUENCE 1234 THREAD 1;
  RESTORE DATABASE;
  RECOVER DATABASE;
  ALTER DATABASE OPEN RESETLOGS;
}
```

Tablespace Point-in-Time Recovery (TSPITR)

Recover specific tablespaces to a point in time while keeping rest of database current.

```
# Basic TSPITR
RMAN> RECOVER TABLESPACE users
```

```
      UNTIL TIME "TO_DATE('2024-02-10 12:00:00', 'YYYY-MM-DD HH24:MI:S
      AUXILIARY DESTINATION '/u01/aux';

# TSPITR with specific files excluded from import
RMAN> RECOVER TABLESPACE users
      UNTIL TIME "SYSDATE-1/24"  -- 1 hour ago
      AUXILIARY DESTINATION '/backup/aux'
      SKIP TABLESPACE temp, undo;
```

TSPITR Notes:

- Requires auxiliary instance automatically created by RMAN
- Objects with cross-tablespace dependencies may be dropped
- Use **DBMS_TTS.TRANSPORT_SET_CHECK** to validate before TSPITR

Block Media Recovery (BMR)

Recover individual corrupt blocks without taking datafiles offline.

```
# Recover specific corrupt blocks
RMAN> RECOVER DATAFILE 5 BLOCK 23, 145, 167;

# Recover all corrupt blocks (RMAN scans for corruption)
RMAN> RECOVER CORRUPTION LIST;

# Find corrupt blocks first
RMAN> VALIDATE DATAFILE 5;
RMAN> SELECT * FROM V$DATABASE_BLOCK_CORRUPTION;
```

Control File Management

Control files contain critical database metadata. Loss of all control files requires special recovery procedures.

Backing Up Control Files

```
# Automatic with every backup (recommended setting)
RMAN> CONFIGURE CONTROLFILE AUTOBACKUP ON;
RMAN> CONFIGURE CONTROLFILE AUTOBACKUP FORMAT FOR DEVICE TYPE DISK TO
      '/backup/%F';

# Manual control file backup
RMAN> BACKUP CURRENT CONTROLFILE;

# Backup control file with tag
RMAN> BACKUP CURRENT CONTROLFILE TAG 'BEFORE_UPGRADE';

# Create control file copy (image copy)
RMAN> BACKUP AS COPY CURRENT CONTROLFILE TO '/backup/control_copy.ctl'

# SQL method for trace file (for manual recreation)
SQL> ALTER DATABASE BACKUP CONTROLFILE TO TRACE AS
     '/backup/control.sql';
```

Restoring Control Files

```
# Scenario 1: One control file lost (simple OS copy)
$ cp /u01/oradata/ORCL/control01.ctl /u01/oradata/ORCL/control02.ctl

# Scenario 2: All control files lost - restore from autobackup
$ rman target /
RMAN> STARTUP NOMOUNT;
RMAN> SET DBID 1234567890;  -- Your database DBID
RMAN> RESTORE CONTROLFILE FROM AUTOBACKUP;
RMAN> ALTER DATABASE MOUNT;
RMAN> RESTORE DATABASE;
RMAN> RECOVER DATABASE;
RMAN> ALTER DATABASE OPEN RESETLOGS;

# Restore from specific location
RMAN> STARTUP NOMOUNT;
RMAN> RESTORE CONTROLFILE FROM '/backup/control_copy.ctl';
RMAN> ALTER DATABASE MOUNT;

# Find and restore from latest autobackup
RMAN> RESTORE CONTROLFILE FROM AUTOBACKUP
      MAXSEQ 100 MAXDAYS 7;
```

Recreating Control File Manually

```
# Generate control file creation script
SQL> ALTER DATABASE BACKUP CONTROLFILE TO TRACE AS
     '/tmp/recreate_control.sql' RESETLOGS;

# Edit the trace file, then execute
SQL> STARTUP NOMOUNT;
SQL> @/tmp/recreate_control.sql
SQL> RECOVER DATABASE USING BACKUP CONTROLFILE;
SQL> ALTER DATABASE OPEN RESETLOGS;
```

Control File Scenario	Recovery Method	Downtime
One control file lost (others intact)	OS copy from good file	Minimal (seconds)
All control files lost (autobackup available)	RMAN RESTORE CONTROLFILE FROM AUTOBACKUP	Minutes
All control files lost (no autobackup)	Recreate from trace file	Hours (manual process)

Flashback Technologies

Flashback features provide logical recovery options without traditional restore/recover operations.

Flashback Database

Rewinds the entire database to a previous point in time using flashback logs.

```
# Enable Flashback Database
SQL> ALTER DATABASE FLASHBACK ON;
SQL> ALTER SYSTEM SET DB_FLASHBACK_RETENTION_TARGET=4320;   -- 3 days i
```

```
# Set Fast Recovery Area size
SQL> ALTER SYSTEM SET DB_RECOVERY_FILE_DEST_SIZE=50G;
SQL> ALTER SYSTEM SET DB_RECOVERY_FILE_DEST='+FRA';

# Check flashback status
SQL> SELECT flashback_on, oldest_flashback_scn, oldest_flashback_time
     FROM v$database;

# Flashback database to timestamp
RMAN> SHUTDOWN IMMEDIATE;
RMAN> STARTUP MOUNT;
RMAN> FLASHBACK DATABASE TO TIME
      "TO_DATE('2024-02-10 14:00:00', 'YYYY-MM-DD HH24:MI:SS')";
RMAN> ALTER DATABASE OPEN RESETLOGS;

# Flashback to SCN
RMAN> FLASHBACK DATABASE TO SCN 2894000;
RMAN> ALTER DATABASE OPEN RESETLOGS;

# Flashback to restore point
SQL> CREATE RESTORE POINT before_upgrade GUARANTEE FLASHBACK DATABASE;
RMAN> FLASHBACK DATABASE TO RESTORE POINT before_upgrade;
RMAN> ALTER DATABASE OPEN RESETLOGS;
```

✅ **Flashback Database Benefits:**

- Much faster than restore/recover (minutes vs hours)
- Can flashback past OPEN RESETLOGS (12c+)
- Great for testing upgrades, patches, or major changes

Flashback Table

Rewind individual tables to a previous state without affecting other objects.

```
# Enable row movement (required for flashback table)
SQL> ALTER TABLE employees ENABLE ROW MOVEMENT;

# Flashback table to timestamp
SQL> FLASHBACK TABLE employees
     TO TIMESTAMP (SYSTIMESTAMP - INTERVAL '15' MINUTE);
```

```
# Flashback to SCN
SQL> FLASHBACK TABLE employees TO SCN 2894000;

# Flashback to before specific operation
SQL> FLASHBACK TABLE employees
     TO BEFORE DROP;  -- Recover dropped table

# Rename table during flashback recovery
SQL> FLASHBACK TABLE employees
     TO BEFORE DROP RENAME TO employees_recovered;
```

Flashback Query

```
# Query data as it existed at a specific time
SQL> SELECT * FROM employees
     AS OF TIMESTAMP (SYSTIMESTAMP - INTERVAL '2' HOUR)
     WHERE department_id = 50;

# Compare current and past data
SQL> SELECT e1.employee_id, e1.salary current_salary, e2.salary old_sa
     FROM employees e1,
          employees AS OF TIMESTAMP (SYSTIMESTAMP - INTERVAL '7' DAY)
     WHERE e1.employee_id = e2.employee_id
       AND e1.salary != e2.salary;

# Recover deleted rows
SQL> INSERT INTO employees
     SELECT * FROM employees
     AS OF TIMESTAMP (SYSTIMESTAMP - INTERVAL '1' HOUR)
     WHERE employee_id = 12345;
```

Flashback Transaction

```
# Find transaction ID from flashback query
SQL> SELECT versions_xid, versions_operation, salary
     FROM employees
     VERSIONS BETWEEN TIMESTAMP
          (SYSTIMESTAMP - INTERVAL '2' HOUR) AND SYSTIMESTAMP
     WHERE employee_id = 100;
```

```
# Flashback (undo) specific transaction
SQL> FLASHBACK TRANSACTION '0A00150023000000';

# Flashback with dependencies
SQL> FLASHBACK TRANSACTION '0A00150023000000'
     CASCADE;  -- Also backs out dependent transactions
```

Flashback Feature	Use Case	Requirements
Flashback Database	Rewind entire database quickly	Flashback logs, FRA
Flashback Table	Undo changes to specific table	Row movement enabled, undo retention
Flashback Query	View historical data, compare changes	Undo retention
Flashback Drop	Recover dropped tables	Recycle bin enabled
Flashback Transaction	Undo specific DML operations	Supplemental logging

Best Practices & Tips

Backup Strategy

- Use **incremental updated image copies** for critical databases requiring minimal downtime
- Enable **control file autobackup** for all databases
- Configure **retention policy** based on business requirements
- Test restore and recovery procedures regularly

- Document your DBID and store it securely

Common Pitfalls

- ✕ Forgetting to back up control files separately
- ✕ Not testing PITR procedures before emergencies
- ✕ Insufficient Fast Recovery Area size for flashback logs
- ✕ Not documenting SCN or timestamps before major changes
- ✕ Ignoring VALIDATE checks before actual restores

Quick Reference Commands

```
# Check backup status
RMAN> LIST BACKUP SUMMARY;
RMAN> LIST COPY;

# Verify backups are usable
RMAN> RESTORE DATABASE VALIDATE;

# Find database ID (critical for recovery)
SQL> SELECT dbid FROM v$database;

# Check current SCN
SQL> SELECT current_scn FROM v$database;

# View archived logs
RMAN> LIST ARCHIVELOG ALL;

# Check flashback capability
SQL> SELECT oldest_flashback_time, retention_target
     FROM v$flashback_database_log;
```

 RMAN Recovery Manager Guide | Oracle Database Backup & Recovery

Always test recovery procedures in non-production environments first!